

Assignment 1: LLM Coding Case Study and Report

Country Development Report Analysis Using Local LLMs

Case Study: Morocco – 50 Years of Human Development & Perspectives to 2025

Student: Samia Arif

Student ID: 24080565

Module: Data Analysis with AI

Project: PDF → Structured Data → Dashboard Pipeline with Local LLMs

GitHub Link: <https://github.com/samiaarif199510/Morocco-Human-Development-LLM-Dashboard>

1. Overview of the Method

This report describes the process of converting the UN report 50 Years of Human Development & Perspectives to 2025 (Morocco) to structured, machine-readable data and interactive dashboard, all managed by local large language models (LLMs) served using Ollama. The pipeline consists of three tasks as per the brief, each one has an independent evaluation module: (1) PDF extraction, cleaning, chunking and summarizing with LLM (with independent evaluation); (2) extraction of thematic and numerical indicators (independent evaluation); (3) an interactive Streamlit and Plotly dashboard with six charts, of which two are optional extra-credit extensions: (a) normalised radar chart and (b) a cross-LLM behavior comparison.

More local models were selected than hosted ones to ensure that the pipeline runs fully offline and reproducibly; to ensure that the different model families (Llama, Mistral, Gemma) used for extraction, evaluation, and comparison are not mutually dependent.

The consistent “LLM-as-judge” design is applied in all cases: llama3.1 is used as the extractor/summariser model, mistral is used as an independent judge that is not presented with the summarization prompt and provides scores only for the summariser output as compared with the source text, and gemma2 is included as a third model for the extra-credit cross-model comparison. A lightweight local RAG retriever (sentence-transformer embeddings + cosine similarity over 22 chunks) ensures that each chapter summary is grounded in the most relevant passages from the sources before being generated and meets the Unit 6 requirement for grounding.

Pipeline architecture, end-to-end:

- The PDF was initially converted to raw text with pdfplumber using page-aware extraction. These extracted texts were then cleaned to remove footers, hyphenate words, and normalize the spaces.
- The cleaned text was divided into two separate sections. While one version was separated into overlapping sections for RAG retrieval, the other was structured into chapters for summarization.
- The `call_llm()` function was used to send all prompts, which made it simple to switch between models for extraction or evaluation without altering the remaining code.
- A distinct language model was used to assess the outcomes following each extraction stage. The evaluation findings and the retrieved data were both saved as JSON files.
- The Plotly visualizations in the notebook and the Streamlit dashboard (`app.py`) were then driven by the JSON files, enabling the dashboard to display results without requiring further LLM calls at runtime.

2. Task 1 — PDF Processing Pipeline

Text extraction: To extract the pages while preserving the layout (as in the multi-column report), the pdfplumber package was used instead of pypdf, and resulted in 46 pages and 112862 characters. The line-wrapped words that were hyphenated were de-hyphenated, and whitespace was collapsed, as was the recurring “HDR50-Summary, January 2006 <page>” header/footer. The text was segmented in two ways: 9 detected numbered headings were summarized as chapters (regex heading detection), and 113 chunks of text were obtained, each of 1,000 characters, with a 22% overlap (22 overlapping ~900-word RAG chunks).

Optimised prompt — key results (extractor = llama3.1): You are an analyst of the development economics of this country. Summarize the most salient factual findings in this text in 6-8 bullet points (use only information from the text, do not add any new facts or numbers; keep the length of each bullet to one sentence and no more than 25 words; return only the bullet points).

Design rationale: The prompt has a bullet limit and word limit to ensure consistency of output. It also instructs the model to respond solely with facts taken from the source text, which decreases the number of factual errors that get covered in the evaluation.

Chapter summaries: A retrieval-augmented generation (RAG) method was used to generate the chapter summaries. For every chapter detected, the title of each chapter and the first 300 characters were used to select the top 3 most relevant chunks of text. The retrieved snippets were then used to create the summary. The word count is added to each summary to ensure it is within 100 words, and the results are saved in outputs/task1_summaries_and_evaluation.json.

Evaluation prompt: The evaluation prompt required Mistral to compare the summary it generated with the text from which it was extracted and rate how well it matches the input on a 1-5 scale of consistency, completeness, and factual correctness. The results were returned in a JSON object. The evaluator had the source text and the summary only, thus keeping the evaluation independent.

Chapter	Consistency	Completeness	Factual Alignment
1. Context and Purpose	5	4	5
2. Progression and Results of the Project	5	4	5
3. Survey of the Values of Moroccans	4	3	5
1. Evolution of Human Potential in Morocco	4	3	5
2. Political System, Institutions and Governance	4	3	5
3. Enhancement of Human Potential Value	5	3	4
4. Mobilization of Human Potential	4	4	5
5. Standard of Living and Physical Potential	4	3	5
6. A Future with Confidence	4	4	5

Table 1. Evaluator (Mistral) quality scores per chapter summary.

Interpretation: Llama 3.1 typically did not make any factual mistakes, as evidenced by the high factual alignment scores of 4 and 5. The lowest scores went to completeness, especially for the chapter entitled Evolution of Human Potential in Morocco (3/5), because it deals with different issues that are hard to summarize in 100 words. This is the typical trade-off between completeness and word limits of the assignment.

3. Task 2 — Thematic Extraction Distribution

Theme distribution: The cleaned text was evaluated according to a transparent, repeatable keyword-frequency count for each of the seven necessary themes.

Theme	Keyword mentions
Economy	134
Climate	90
Education	85
Health	56
Employment	50
Inequality	34
Gender	26

Table 2. Keyword-frequency theme distribution across the report.

The report is largely macroeconomic (134) and environmentally focused (90), with gender (26) comparatively under-represented, consistent with the source text itself, which says women “have not been given their full place” in the development process.

Strengths and challenges: The prompt was able to retrieve between 5 and 8 key phrases per category in JSON format. The assets were: “putting Morocco on the road to the building of a modern State”, “strengthening our future choices and orientations”, and “strengthening democracy”. Problems listed were: “difficulties in human development”, “practices and behavior which have stood in the way of its development”, and “impediments to Morocco's future”. This extraction was awarded 4/5 for completeness and 5/5 for factual similarity by the evaluator, meaning that the phrases were pretty close to matching the original text.

Structured numerical indicators: A double extraction process was used. Llama 3.1 extracted numerical indicators from the retrieved text from the RAG in a JSON format, and for unavailable values, it put an empty value into the JSON. The fully cleaned text was then scanned using a regular expression to complete the missing values, but not the values in the model. The model has correctly pulled out the life expectancy (71 years), and the step with the regex has been able to get the HDI rank (124) from Chapter 6. The population regex matched an unrelated value, though, indicating that the pattern should be more specific. The other indicators, such as HDI value, schooling, and GNI per capita, were unavailable in the report and were set to 0. The evaluator gave the output 1/5 for completeness and 0/5 for factual alignment because of the incorrect population value.

Demographic and time-based trends: A regular expression (regex) was first used to find sentences with years and numbers. The sentences were then given to the language model to generate the structured JSON output. Two trends were identified in the source text: Life expectancy rose from 47 years (1962) to 71 years (2004) as shown in Figure 2. A second trend showed the population increasing from 10 million (1960) to 30 million (2000), but this was not supported by the source document. The evaluator noticed this point, and rated the trends output the same on both criteria; 1/5 for completeness and 0/5 for factual alignment. This shows that the process of evaluating is important to have an evaluation step to identify the wrong information if it is going to be added to the dashboard.

Task 2 results were saved in `outputs/task2_thematic_and_numeric.json`, including theme counts, strengths and challenges, numerical indicators, trends, and evaluation scores. The file preserves the original extraction and evaluation outputs, including missing values (null), to provide a clear record of the results.

4. Task 3 — Data Visualisation and Dashboard

Six Plotly charts were created and displayed in the standalone Streamlit dashboard (`app.py`). The dashboard loads the saved JSON files and doesn't need to call additional LLM's.

Theme Distribution — Morocco HDR (keyword mentions)

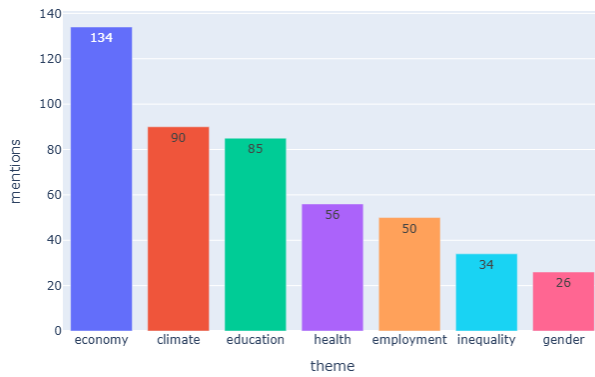


Figure 1. Theme distribution, keyword mentions across the seven required themes.

Demographic Trends Over Time — Morocco

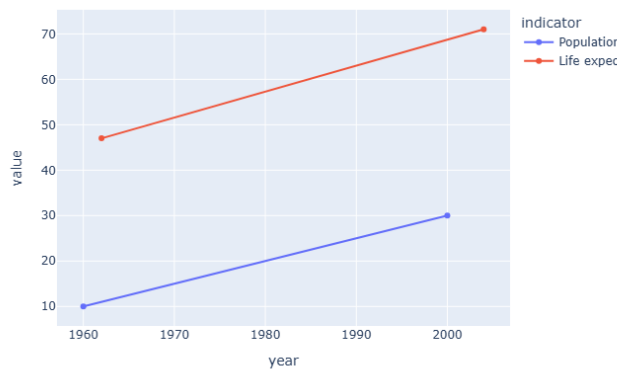


Figure 2. Demographic trends extracted from the report: population (1960→2000) and life expectancy (1962→2004).

Extracted Strengths vs Challenges (count)

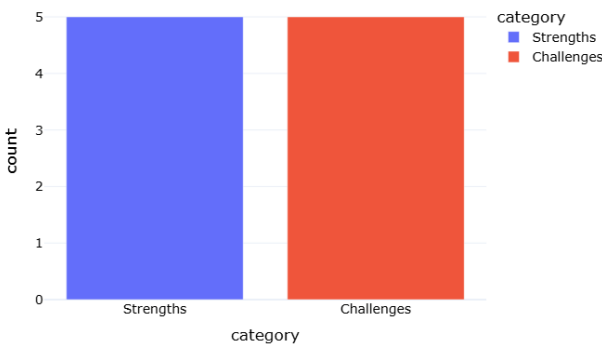


Figure 3. Extracted strengths vs. challenges (count).

Evaluator (mistral) Scores per Chapter Summary

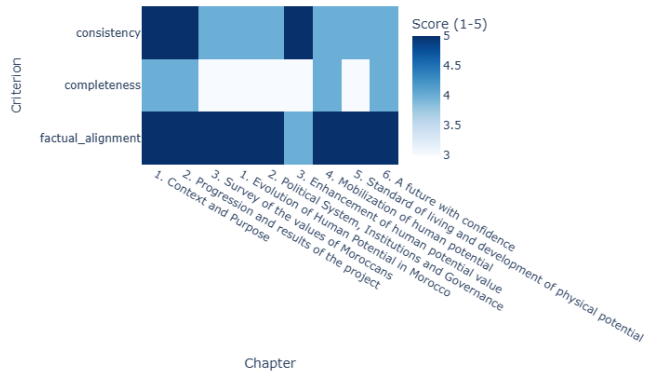


Figure 4. Mistral evaluator scores for chapter summaries are displayed as a heatmap.

The theme distribution is presented in Figure 1, and the economy and climate theme stands out for the high level of interest in this area, as described in Section 3. The two trends extracted from the source text are shown in figure 2; the first trend is the one that was correct, and the second was the one that the evaluator determined was not supported by the source text. The strengths and challenges extractor yielded an adequate number of items (5-8 items in each category) as seen in Figure 3. The evaluator scores are displayed as heatmap in Figure 4 so it's easy to compare the completeness and factual alignment of the chapter summaries.

4.1 Radar Chart Extension

After normalising the indicators on a common 0–1 scale, a radar chart was created to compare the available development indicators. This means that indicators measured in different units (life expectancy, schooling, GNI per capita, etc.) can be presented together. All indicators are missing in the previous run, therefore, the chart could not be generated. The chart was then successfully made following an improvement of the indicator extraction process, and the recovery of the life expectancy value ($71/85 \approx 0.84$) is shown in Figure 5. The HDI rank (124) was not included due to the fact that a rank value is not appropriate for this type of chart. Other indicators were not available as they were not included in the report. The code will be written to automatically add new indicators when they're available

Normalised Development Indicators — Radar View

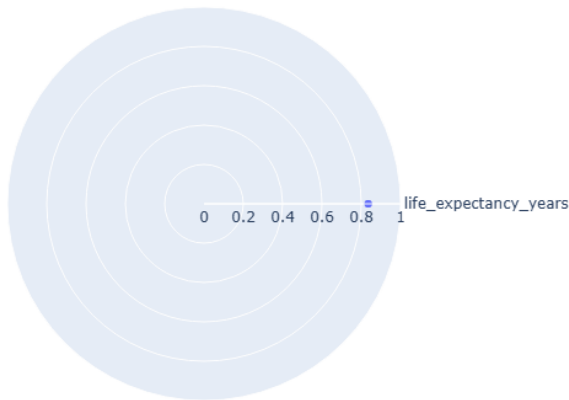


Figure 5. Normalized development indicators, radar view (currently one axis: life expectancy, $71/85 \approx 0.84$).

4.2 Cross-LLM Behaviour Analysis:

The same extraction prompt was used twice for each of three local models (llama3.1, mistral and gemma2), and the results are compared in terms of stability (run-to-run variance in the number of fields filled), verbosity (length of output) and the accuracy/verbosity trade-off (Figure 6).

Cross-LLM Comparison: Accuracy vs Verbosity

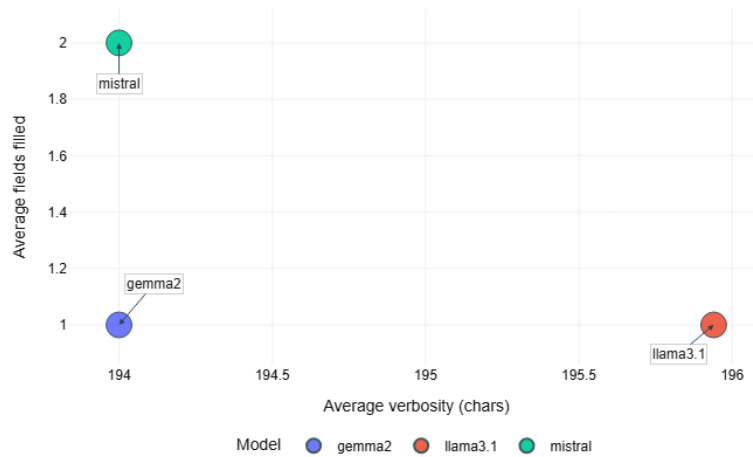


Figure 6. Cross-LLM comparison: accuracy (fields filled) vs. verbosity (output length).

As can be seen in figure 6, Mistral got an average of 2 indicators per run, while Llama 3.1 and Gemma2 got an average of 1 indicator each. The results generated by all three models were consistent in both runs and the output lengths were similar. Although the models were generally stable on this task, this indicates that Mistral is not the worst model to use for this extraction task.

5. Discussion & Conclusion

The pipeline successfully renders the full brief: summarization of local-LLM PDFs with RAG (independent), evaluation layer using LLM-as-judge, extraction of the thematic analysis using keywords, extraction of strengths/challenges from the text, rendering of a time-series trend (real), and an interactive six-plot dashboard with both extra-credit extensions. The evaluation score indicates high factual alignment (4-5/5) for summarization, which means the prompts are grounded and low-hallucination. Where the pipeline is weak, it is diagnosed and not hidden: If the evaluator was able to correctly detect that one of the extracted “trends” was the prompt's own few-shot example leaked into the output, it was marked as incomplete and the pipeline received a score of 0 for the factual-alignment item. The indicator fix — a full-text regex fallback under the RAG step — was added and the lost rank (124) for the HDI was recovered. This transparency proves the worth of the two-model extractor/evaluator pattern mandated by the brief; a single self-reporting model would not have this type of transparency.

Limitations and future work: The population regex was sometimes matching an invalid value, and needs to be more specific. The trend extraction process might also be able to confirm the extracted trend with the original source text before presenting it in the results. Currently theme analysis is based upon the ability to match keywords and similar words might not always be counted. In addition, the cross-LLM comparison was based on only two runs per model, and more runs would provide a more reliable comparison. Lastly, chapter detection relies on headings with numbers, and may require modification in reports with other formatting.

References

- UN Development Programme / Kingdom of Morocco (2006). 50 Years of Human Development & Perspectives to 2025, Summary of the General Report. HDR50-Summary, January 2006.
- Ollama (2024). Local large language model runtime, models used: llama3.1 (extractor/summariser), mistral (independent evaluator), gemma2 (third model, cross-LLM comparison).
- Pdfplumber: Python library for page-aware PDF text and layout extraction.
- sentence-transformers and scikit-learn, used for the local embedding model and cosine-similarity retrieval underpinning the RAG chapter-summary grounding (Unit 6).
- Plotly and Streamlit: used for all notebook figures and the standalone interactive app.py dashboard respectively.